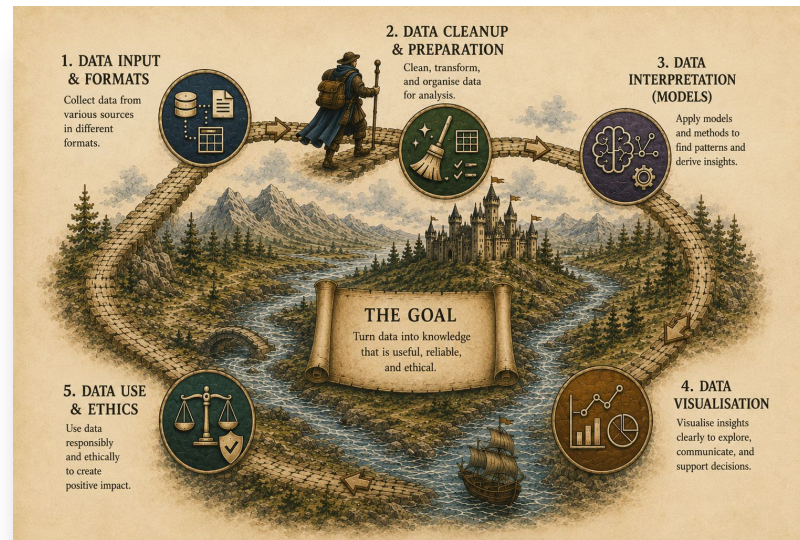


COMP20008 Elements of Data Processing



Lecture 5: Unstructured Data & Processing (“web crawling and scraping”)

Semester 2 2026

Subject Coordinator: **Dr Ekaterina Vylomova**

Co-lecturer: **Dr Hanzhuo (Vivian) Ma**



THE UNIVERSITY OF
MELBOURNE

Announcements

CALL FOR PARTICIPATION

<http://www.alta.asn.au/events/sharedtask2026>

The Australasian Language Technology Association (ALTA) is organising a programming competition for university undergraduate and postgraduate students.

All participants compete to solve the same problem. The problem highlights an active area of research and programming in the area of language technology.

This year's shared task is: **classify the sentiment and the sarcasm of Australian and British English text**

The tentative key dates are:

Right Now - Registration and release of training and development data

22 Sep 2026 - Release of test data

28 Sep 2026 - Deadline of submission of runs

01 Oct 2026 - Notification of results

26 Oct 2025 - Deadline of submission of system description

30 Nov - 2 Dec 2026 - Presentation of results at ALTA 2026

Unstructured data

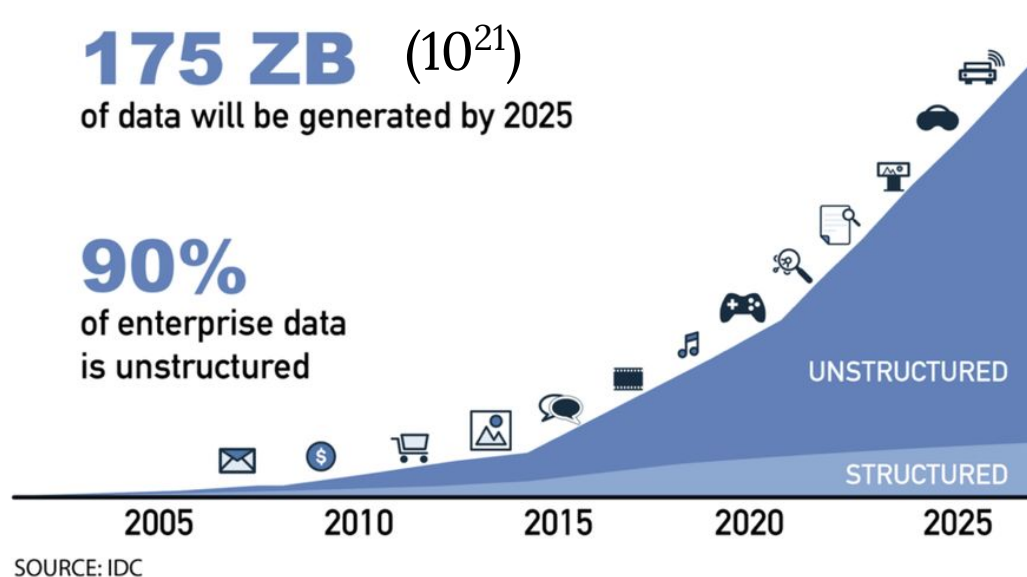


Image source: <https://www.igneous.io/blog/structured-data-vs-unstructured-data>

Unstructured Data is with no (or hidden) structure
Lacks regularity and (easily) decomposable internal structure

Unstructured data

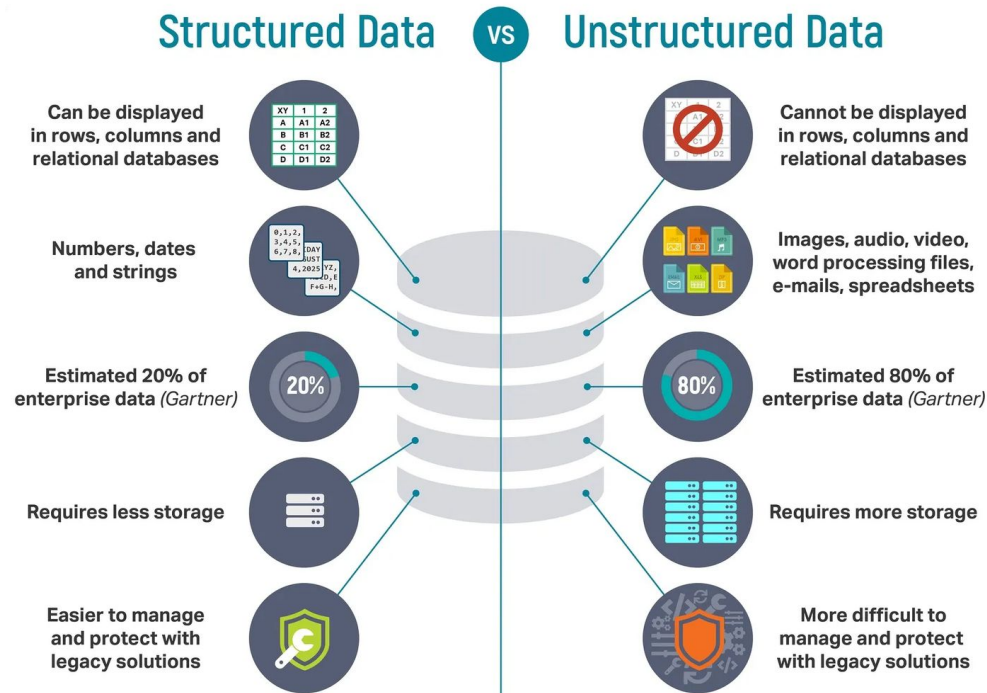
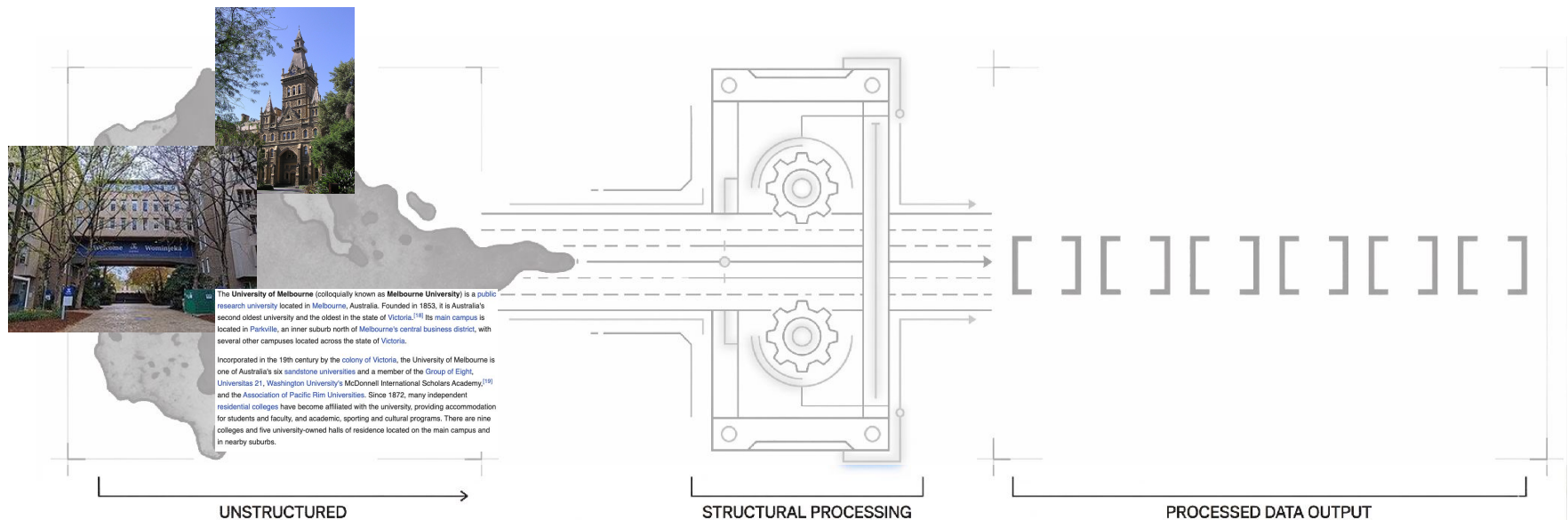


Image source: <https://www.igneous.io/blog/structured-data-vs-unstructured-data>

Unstructured Data is with no (or hidden) structure
Lacks regularity and (easily) decomposable internal structure

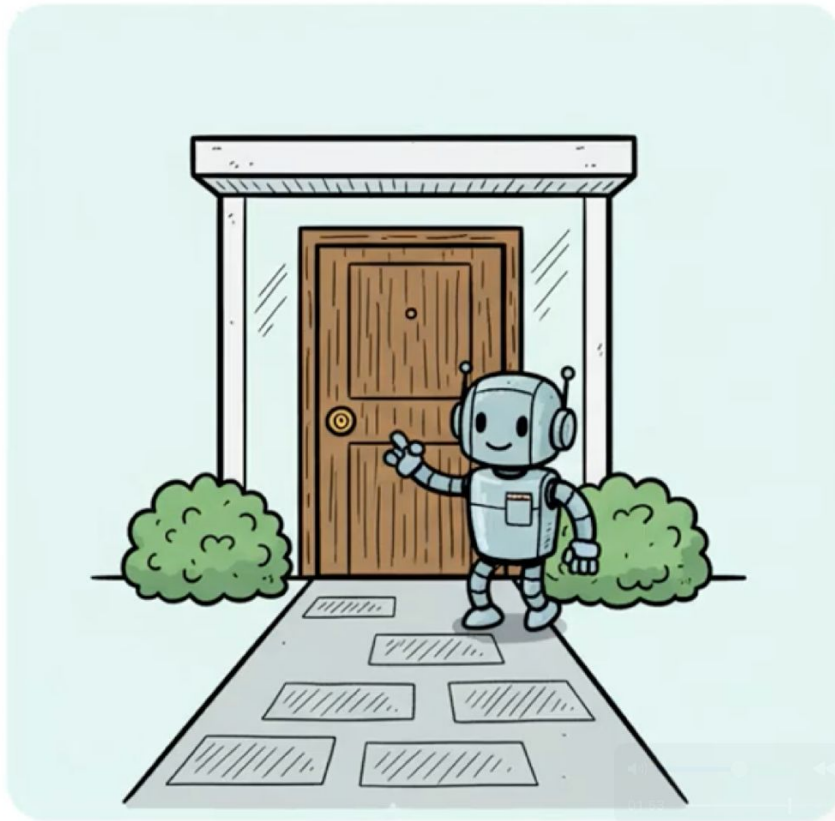
Unstructured data: Text and images



- Need specialised methods to search for and process information
- You must collect it and process/clean it from the web before you can use it

Getting Data from the Web

Web data can be collected in two main ways.



https://en.wikipedia.org/wiki/Natural_language_processing

Natural language processing

🌐 72 languages ▾

Article [Talk](#)

[Read](#) [Edit](#) [View history](#) ⋮

From Wikipedia, the free encyclopedia

This article has multiple issues. Please help [improve it](#) or discuss these issues on the [talk page](#). *(Learn how and when to remove these messages)* [\[hide\]](#)



- This article **needs more citations**. *(May 2024)*
- This article **may need to be rewritten to comply with Wikipedia's quality standards**. *(July 2025)*
- This article **may be in need of reorganization to comply with Wikipedia's layout guidelines**. *(July 2025)*

Natural language processing (NLP) is the processing of [natural language](#) information by a [computer](#). NLP is a subfield of [computer science](#) and is closely associated with [artificial intelligence](#). NLP is also related to [information retrieval](#), [knowledge representation](#), [computational linguistics](#), and [linguistics](#) more broadly. ^[1]

Major processing tasks in an NLP system include: [speech recognition](#), [text classification](#), [natural language understanding](#), and [natural language generation](#).

History [\[edit\]](#)

Further information: [History of natural language processing](#)

Natural language processing has its roots in the 1950s.^[2] Already in 1950, [Alan Turing](#) published an article titled "[Computing Machinery and Intelligence](#)," which

Part of a series on

[Artificial intelligence \(AI\)](#)

[Major goals](#) [\[hide\]](#)

[Artificial general intelligence](#) ·
[Intelligent agent](#) ·
[Recursive self-improvement](#) · [Planning](#) ·
[Computer vision](#) · [General game playing](#) ·
[Knowledge representation](#) ·
[Natural language processing](#) · [Robotics](#) ·
[AI safety](#)

[Approaches](#) [\[show\]](#)

[Applications](#) [\[show\]](#)

[Philosophy](#) [\[show\]](#)

[History](#) [\[show\]](#)

[Controversies](#) [\[show\]](#)

Method 1: Using an API

An **API (Application Programming Interface)** allows programs to request data directly from a service. Instead of downloading an entire webpage, you ask the server for the data you need.

For example,

```
import requests

url =
"https://jsonplaceholder.typicode.com/todos/1"
response = requests.get(url)

data = response.json()
print(data)
```

```
{
  "userId": 1,
  "id": 1,
  "title": "delectus aut autem",
  "completed": false
}
```


Method 1: Using an API

```
import requests

url = "https://en.wikipedia.org/w/api.php"

params = {
    "action": "query",
    "format": "json",
    "formatversion": 2,
    "titles": "Natural language processing",
    "prop": "extracts",
    "exintro": True,
    "explaintext": True
}

headers = {
    "User-Agent": "Kat"
}

response = requests.get(
    url,
    params=params,
    headers=headers,
    timeout=30
)

response.raise_for_status()

data = response.json()

page = data["query"]["pages"][0]

print("Title:", page["title"])
print()
print(page["extract"])
```

```
vylovovae@41801-213247-1:~/Teaching/Elements of Data Processing/2026_S1/Kat's$ python3 api.py
Title: Natural language processing
```

```
Natural language processing (NLP) is the processing of natural language information by a computer. NLP is a subfield of computer science and is closely associated with artificial intelligence. NLP is also related to information retrieval, knowledge representation, computational linguistics, and linguistics more broadly.
```

```
Major processing tasks in an NLP system include: speech recognition, text classification, natural language understanding, and natural language generation.
```

Method 1: Using an API

```
import requests

url = "https://en.wikipedia.org/w/api.php"

params = {
    "action": "query",
    "format": "json",
    "formatversion": 2,
    "titles": "Natural language processing",
    "prop": "extracts",
    "exintro": True,
    "explaintext": True
}

headers = {
    "User-Agent": "Kat"
}

response = requests.get(
    url,
    params=params,
    headers=headers,
    timeout=30
)

response.raise_for_status()

data = response.json()

page = data["query"]["pages"][0]

print("Title:", page["title"])
print()
print(page["extract"])
```

Most data cannot be
access via API!

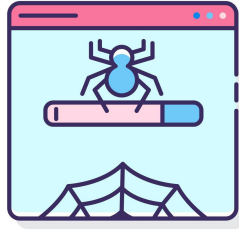
```
vylovovae@41801-213247-1:~/Teaching/Elements of Data Processing/2026_S1/Kat's$ python3 api.py
Title: Natural language processing

Natural language processing (NLP) is the processing of natural language information by a computer. NLP is
a subfield of computer science and is closely associated with artificial intelligence. NLP is also relat
ed to information retrieval, knowledge representation, computational linguistics, and linguistics more br
oadly.
Major processing tasks in an NLP system include: speech recognition, text classification, natural languag
e understanding, and natural language generation.
```

Lecture outline

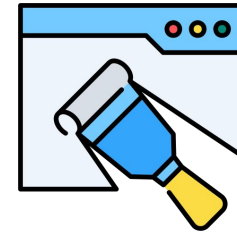
1. **Crawling** for data
2. Document **Scrapping**

Method 2: Crawling and Scraping data



Crawling finds data that you want on the web

- Starts with a **list of seed URLs**
- Crawling **visits all websites** within the linked graph
- Crawling is **untargeted** (it simply records the document(s) it finds)

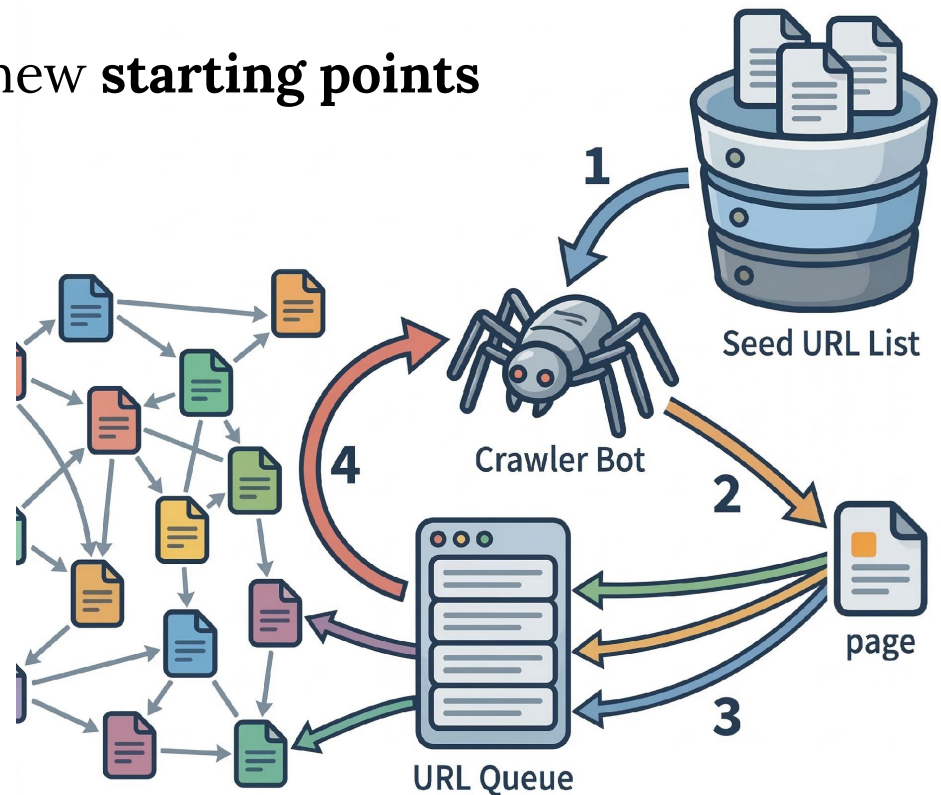


Scraping extracts it from a file you have found

- Scraping starts with a **document** (e.g., a web page)
- Scraping is **the process of extracting** data from a found file
- Scraping is **targeted** (it is given the information that it has to extract)

Crawling the web: HOW?

1. Start with **a list of seed URLs**
2. **Visit** their pages to **collect everything** they point to
3. Use **these new pages** as new **starting points**
4. **Keep looping**



Crawling the web

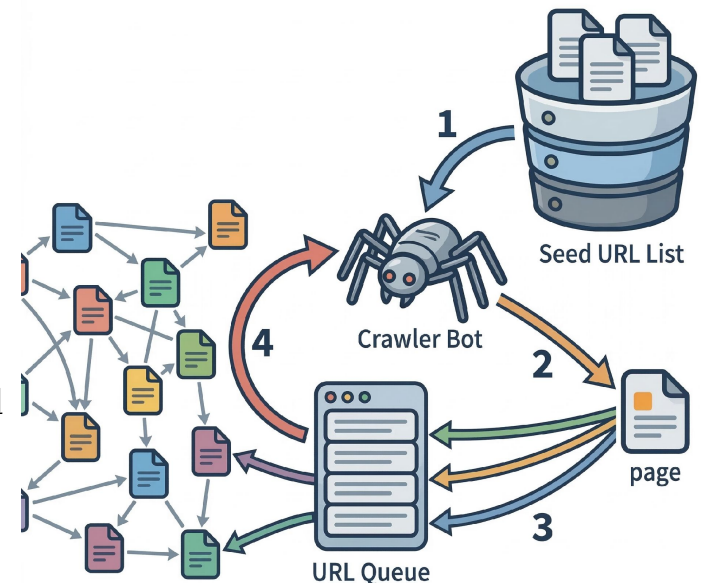
Interesting points

- Web crawlers are known as **spiders, robots, or bots**.
- They attempt to **visit every page of interest** and retrieve them for processing and indexing.
- Duplicate URLs must be avoided
- The crawler must avoid infinite loops.

There is no central index of the URLs you want!

Secondary challenges:

- The old URL may have the same content as the new URL
- A crawler never knows when it has finished. (looping)
- Some websites (spam/doorways/harm) are not intended to be crawled.
- Some content is generated on-the-fly from databases. (e.g. **SNS feeds**)
- Some content has a short lifespan and must be crawled more frequently (e.g. **news articles**)



Crawling the web

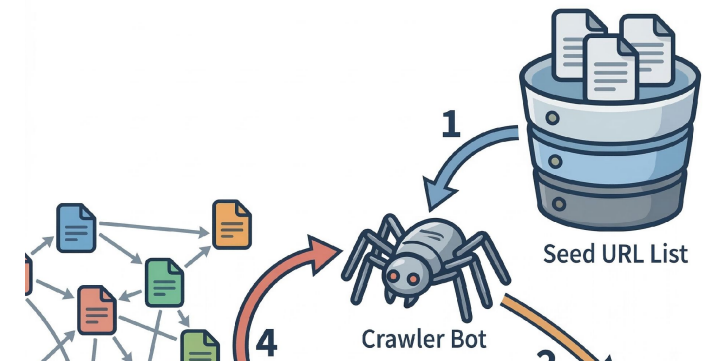
Interesting points

- Web crawlers are known as **spiders, robots, or bots**.
- They attempt to **visit every page of interest** and retrieve them for processing and indexing.
- Duplicate URLs must be avoided
- The crawler must avoid infinite loops.

There is no central index of the URLs you want!

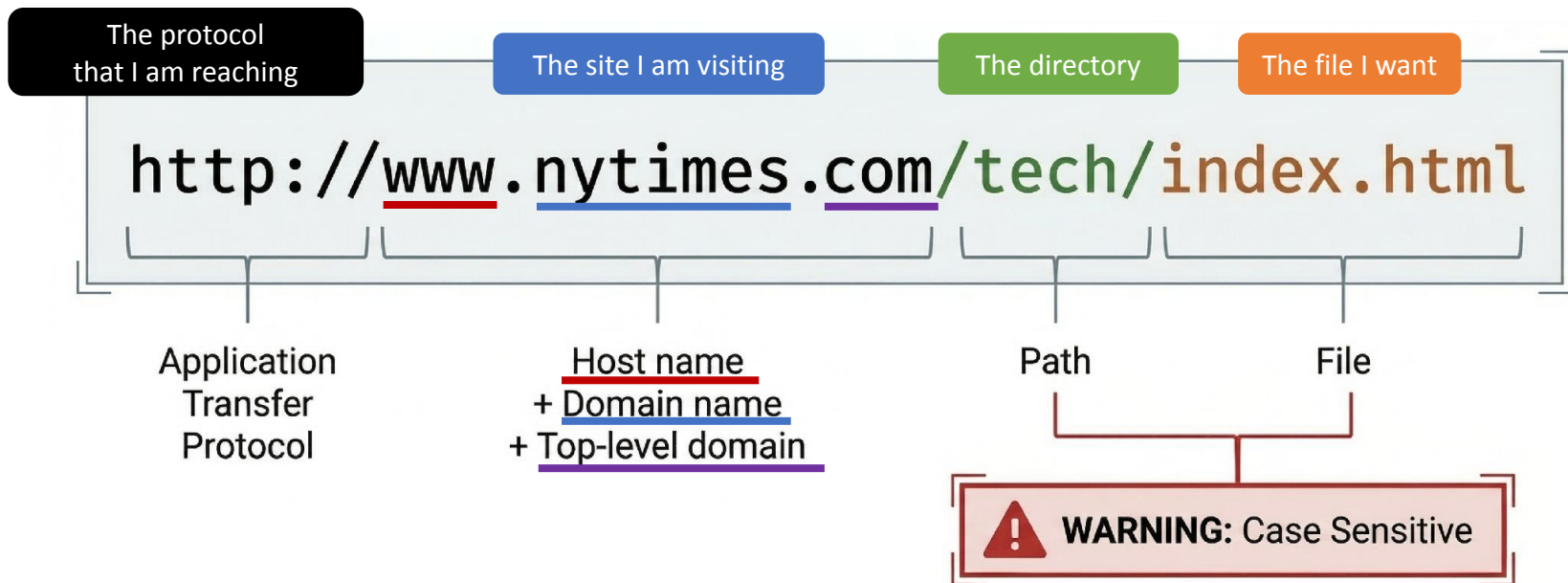
Secondary challenges:

- The old URL may have the same content as the new URL
- A crawler never knows when it has finished. (looping)
- Some websites (spam/doorways/harm) are not intended to be crawled.
- Some content is generated on-the-fly from databases. (**e.g. SNS feeds**)
- Some content has a short lifespan and must be crawled more frequently (**e.g. news articles**)



2009-2011: I worked in a QA team for web crawling (spam identification)

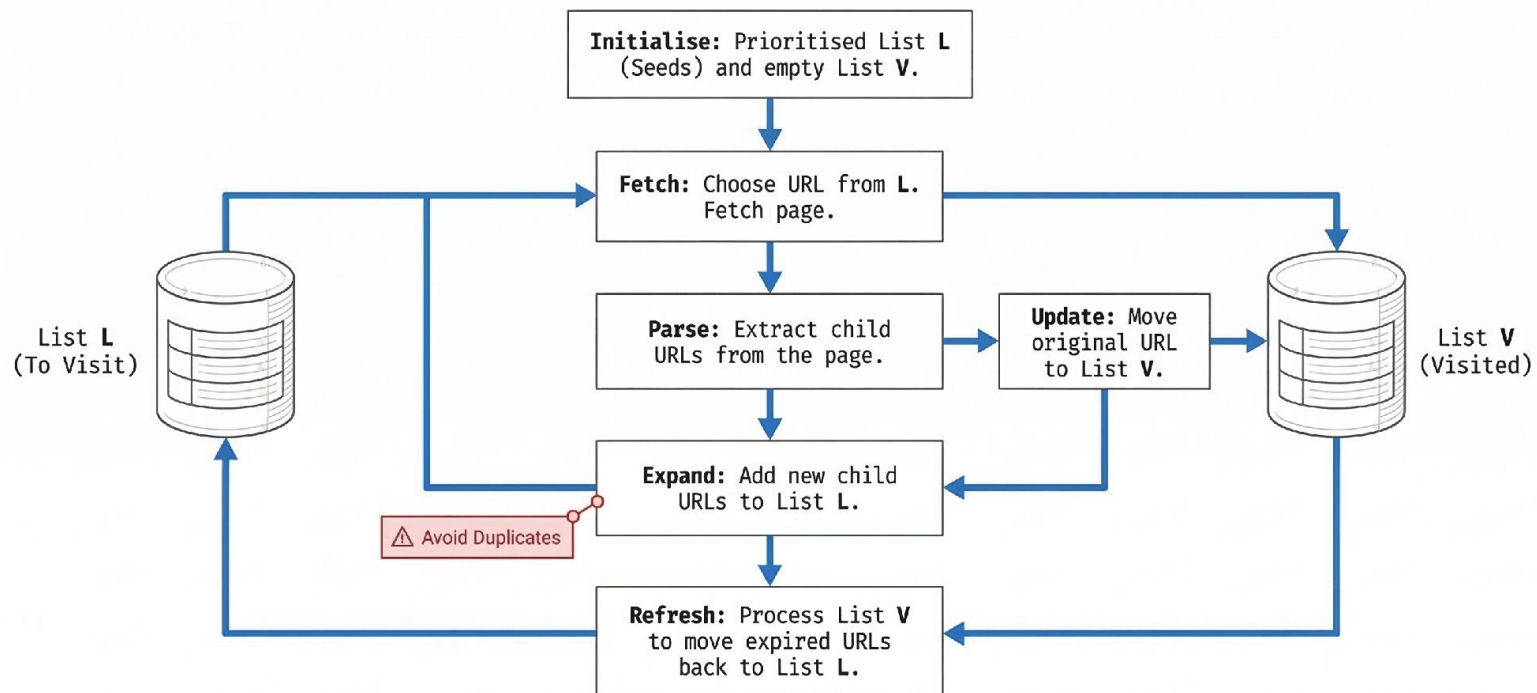
Anatomy of a URL: Target of Crawl



Crawling Algorithm



- The web is a highly linked graph
- If a web page is of interest, there will be a link to it from another page. More important pages will have higher number of links to them (e.g., the idea behind [PageRank](#))



Crawling Algorithm

Repeat Forever:

- Choose a URL u from L and fetch the page $p(u)$ at location u
- Parse and index $p(u)$
Extract URLs $\{u'\}$ from $p(u)$
- Add u to V and remove it from L
- Add $\{u'\} - V$ to L
 - Add them at the front: depth-first search
 - Add them at rear: breadth-first search
 - Rank and insert: best-first searchAVOID DUPLICATES!
- Process V to move expired or 'old' URLs to L

Crawling Algorithm

Repeat Forever:

- Choose a URL u from L and fetch the page $p(u)$ at location u
- Parse and index $p(u)$
Extract URLs $\{u'\}$ from $p(u)$
- Add u to V and remove it from L
- Add $\{u'\} - V$ to L
 - Add them at the front: depth-first search
 - Add them at rear: breadth-first search
 - Rank and insert: best-first searchAVOID DUPLICATES!
- Process V to move expired or 'old' URLs to L

Common Crawl	
	COMMON CRAWL
Type of business	501(c)(3) non-profit
Founded	2007
Headquarters	San Francisco, California; Los Angeles, California, United States
Founder	Gil Elbaz
Managing director	Rich Skrenta
URL	commoncrawl.org
Content license	Apache 2.0 (software) [clarification needed]

<https://commoncrawl.org/>

Navigating Hazards & Spider Traps

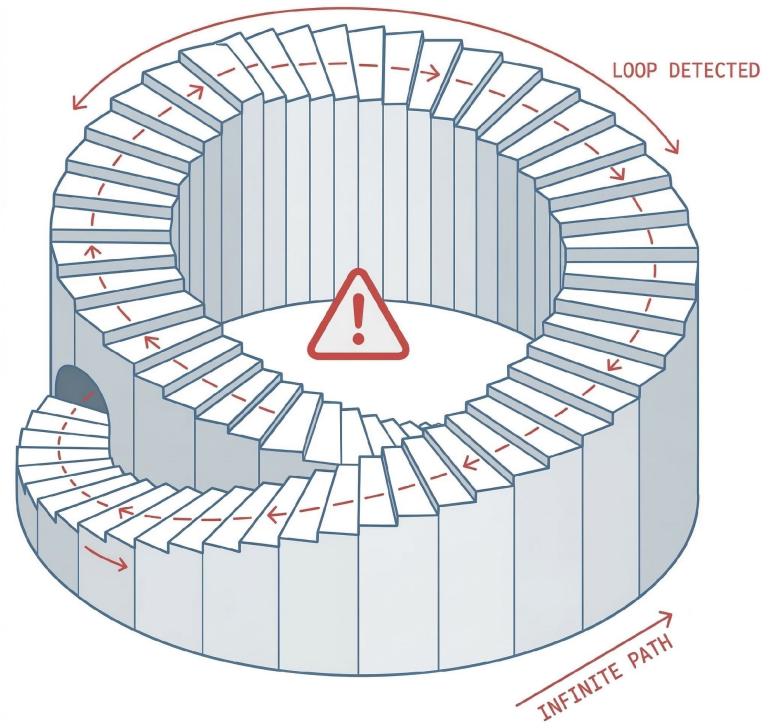
The web is **not a static document repository**. Even non-malicious pages present severe architectural challenges for automated crawlers.

Spider Traps: Infinite, dynamically generated links (e.g., clicking "next month" on a digital calendar infinitely).
Causes crawlers to cycle forever.

Duplicate Pages: Synonym URLs and site mirrors inflate the queue with redundant data.

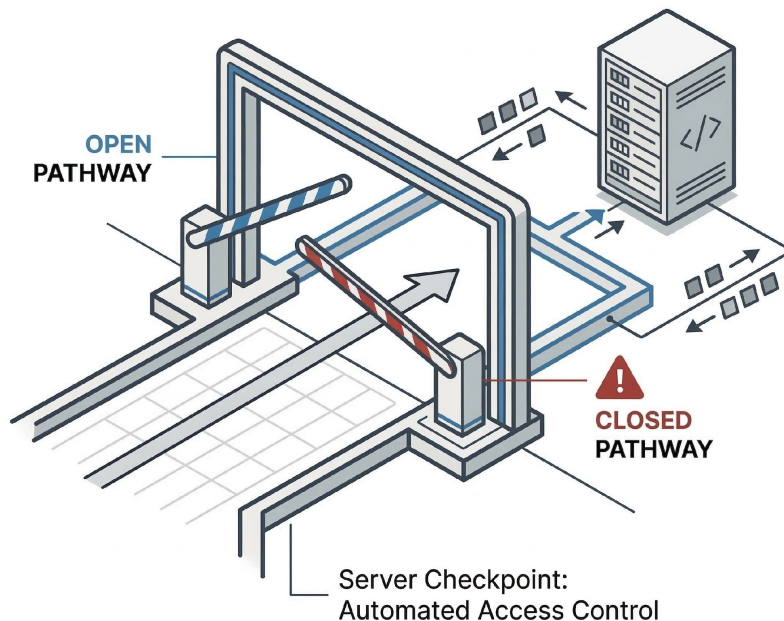
Dynamic Generation: Content generated on-the-fly from databases costs the provider money, making excessive crawler visits highly unwelcome.

Latency Variability: Bandwidth and response times from remote servers are unpredictable.



The Gateway: The Robots Exclusion Standard

Because Google does so much crawling, they developed a useful standard so you can tell them what they should and should not crawl: **robots.txt**



Fields supported by Googlebot

user-agent: The ID check. identifies which crawler the rules apply to

allow: a URL path that may be crawled

disallow: a URL path that may **not** be crawled

sitemap: the complete URL of a sitemap

By adding your sitemap to your **robots.txt** you are making it easier for a bot to find all the links on your site

robots.txt – examples

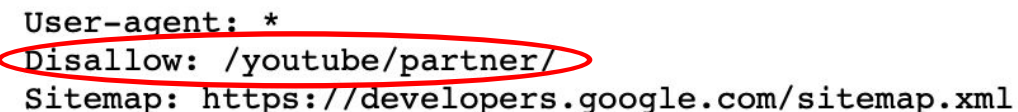
You can check those by yourself!

<https://www.unimelb.edu.au/robots.txt>



```
User-agent: *
Disallow: /home-new/
Disallow: /design/
Disallow: /staffdev/
Disallow: /_data/assets/word_doc/0011/2070866/Privacy-Impact-Assessment-Template-V8
Disallow: /_assets/
Disallow: /templates/
Disallow: /template-assets/
Disallow: /template-assets-custom/
Disallow: /templates-stuff/
https://www.unimelb.edu.au/sitemaps/primary-sitemap.xml
```

<https://developers.google.com/robots.txt>



```
User-agent: *
Disallow: /youtube/partner/
Sitemap: https://developers.google.com/sitemap.xml
```

The Crawler's Code of Conduct

MUST DO (Compliance)

Be Polite: Only crawl allowed pages; strictly respect robots.txt and explicit politeness delays.

Be Legal: Respect copyright, intellectual property, and ownership boundaries.

Be Robust: Engineer immunity to spider traps and malicious server behaviours.

SHOULD DO (Performance)

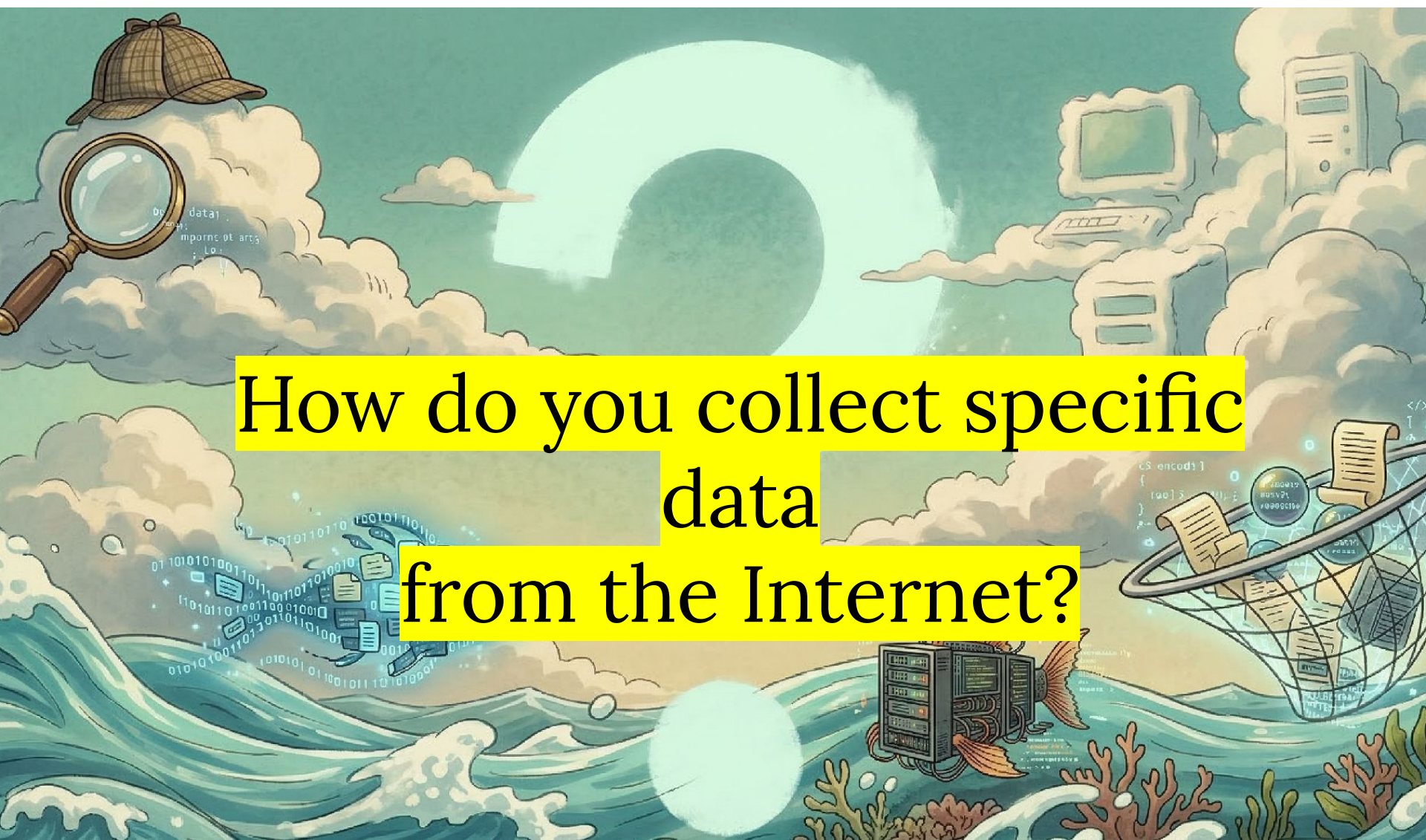
Be Parsimonious: Avoid fetching duplicated web pages to save bandwidth.

Be Efficient: Fetch high-quality pages first; re-fetch to ensure fresh copies.

Be Scalable: Run across multiple machines to increase the overall crawl rate.

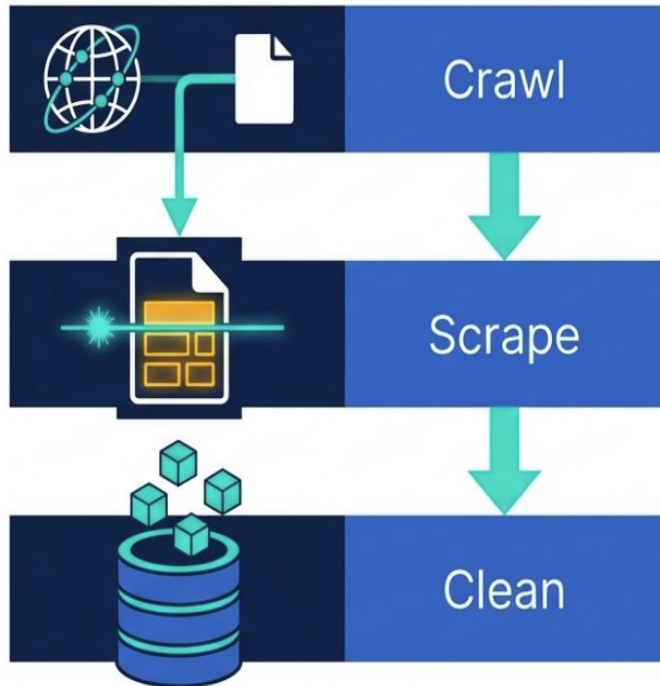
Lecture outline

1. **Crawling** for data
2. Document **Scrapping**



How do you collect specific
data
from the Internet?

The Three-Stage Scraping Pipeline



Crawl the web for documents

Get the HTML or XML documents

Scrape info out of the documents

Extract targeted information using standard routines.

*Note: HTML and XML are already halfway structured, so we use libraries like **Beautiful Soup** or **lxml**.*

Clean and save the information

Process pure “raw” text using **NLP techniques** to ensure standardisation.

Scraping Basics

- Scraping starts with a document
- Scraping is the process of extracting data
- Scraping is targeted (= you give the algorithm the information that it must extract)
- You also specify the eventual format you want the scraped data to have

Cleaning text markup

- You need ‘clean’ raw text for many AI / Natural Language Processing (NLP) applications — no markup, no whitespace, etc.
 1. Remove and handle the file preamble
 2. Compress extra whitespace
 3. Strip out unnecessary XML or HTML or other markup
 4. But what to do with readable markup?
- Questions:
 - What to do with itemized bullets?
 - What to do with empty lines?
- Options:
 1. Write a markup processor
 2. Use a software package

Once upon a time there were three sisters; and their names were

- Elsie,
- Lacie and
- Tillie.

They lived at the bottom of a well.

?

Once upon a time there were three sisters; and their names were Elsie, Lacie and Tillie. They lived at the bottom of a well.

Scraping HTML/XML in Python

The hard way:

1. Import the XML file into your program as a tree structure:

```
import xml.etree.ElementTree as ET
tree = ET.parse('yourfile.xml')
root = tree.getroot()
```

2. Then loop through elements under the root and apply the various relevant methods:

```
for child in root:
    print child.tag, child.attrib
```

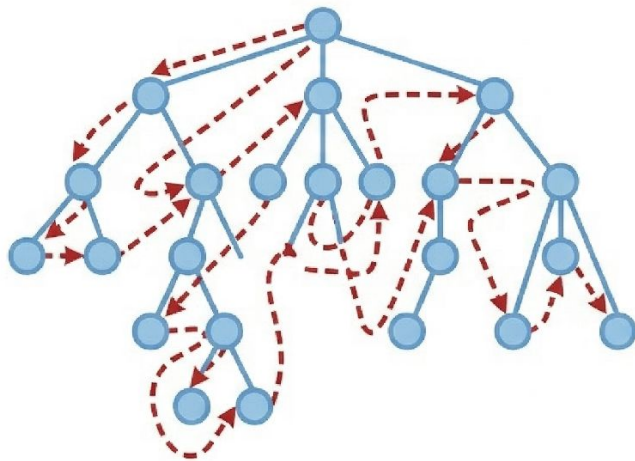
The easy way:

Just use a software package

Scraping HTML/XML in Python

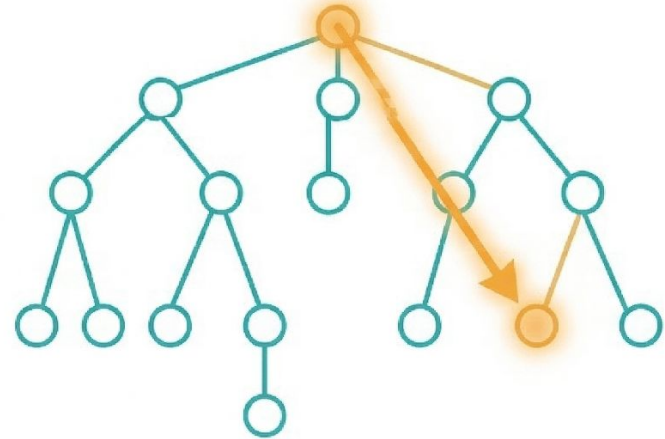
The hard way

```
import xml.etree.ElementTree as ET
root = tree.getroot()
for child in root:
    print child.tag
```



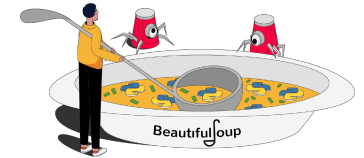
The easy way

```
from bs4 import BeautifulSoup
soup = BeautifulSoup(html, 'html.parser')
```



Beautiful Soup: the easy way

Software package that strips out html/xml and allows you to easily find (parts of) the text



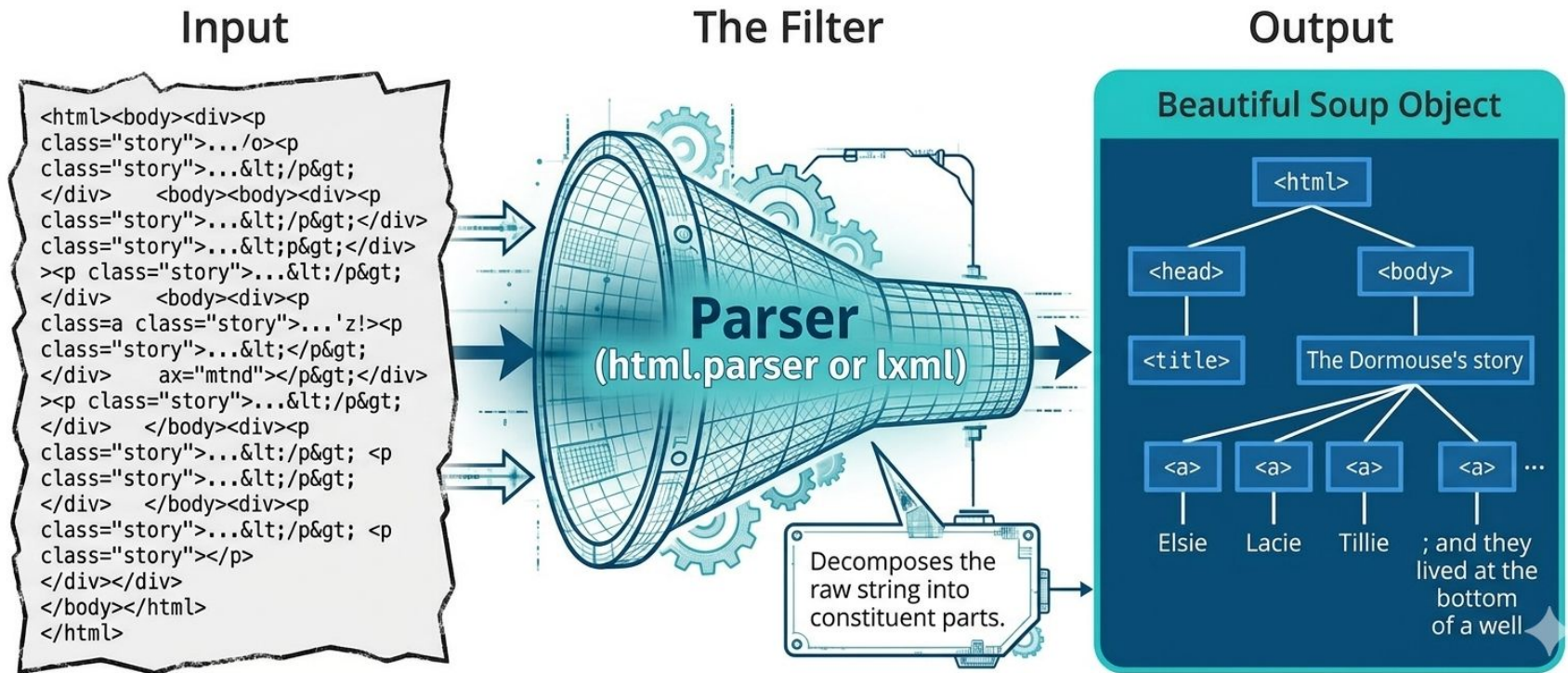
Steps:

1. First, apply a 'parser':
a program that decomposes an HTML/XML document into its parts
2. Then, run BeautifulSoup to extract the pieces you want

Beautiful Soup parsers:

- **HTML input:** parser included in Python's standard library, but also other third-party Python parsers
 - **XML input:** lxml parser
-
- More at <https://beautiful-soup-4.readthedocs.io/en/latest/>

Beautiful Soup: the easy way



Beautiful Soup example – 1

Input displayed Once upon a time there were three little sisters; and their names were [Elsie](#), [Lacie](#) and [Tillie](#); and they lived at the bottom of a well.

Input in HTML

```
html_doc = """
<html><head><title>The Dormouse's story</title></head>
<body>
<p class="title"><b>The Dormouse's story</b></p>

<p class="story">Once upon a time there were three little sisters; and their names were
<a href="http://example.com/elsie" class="sister" id="link1">Elsie</a>,
<a href="http://example.com/lacie" class="sister" id="link2">Lacie</a> and
<a href="http://example.com/tillie" class="sister" id="link3">Tillie</a>;
and they lived at the bottom of a well.</p>

<p class="story">...</p>
"""
```

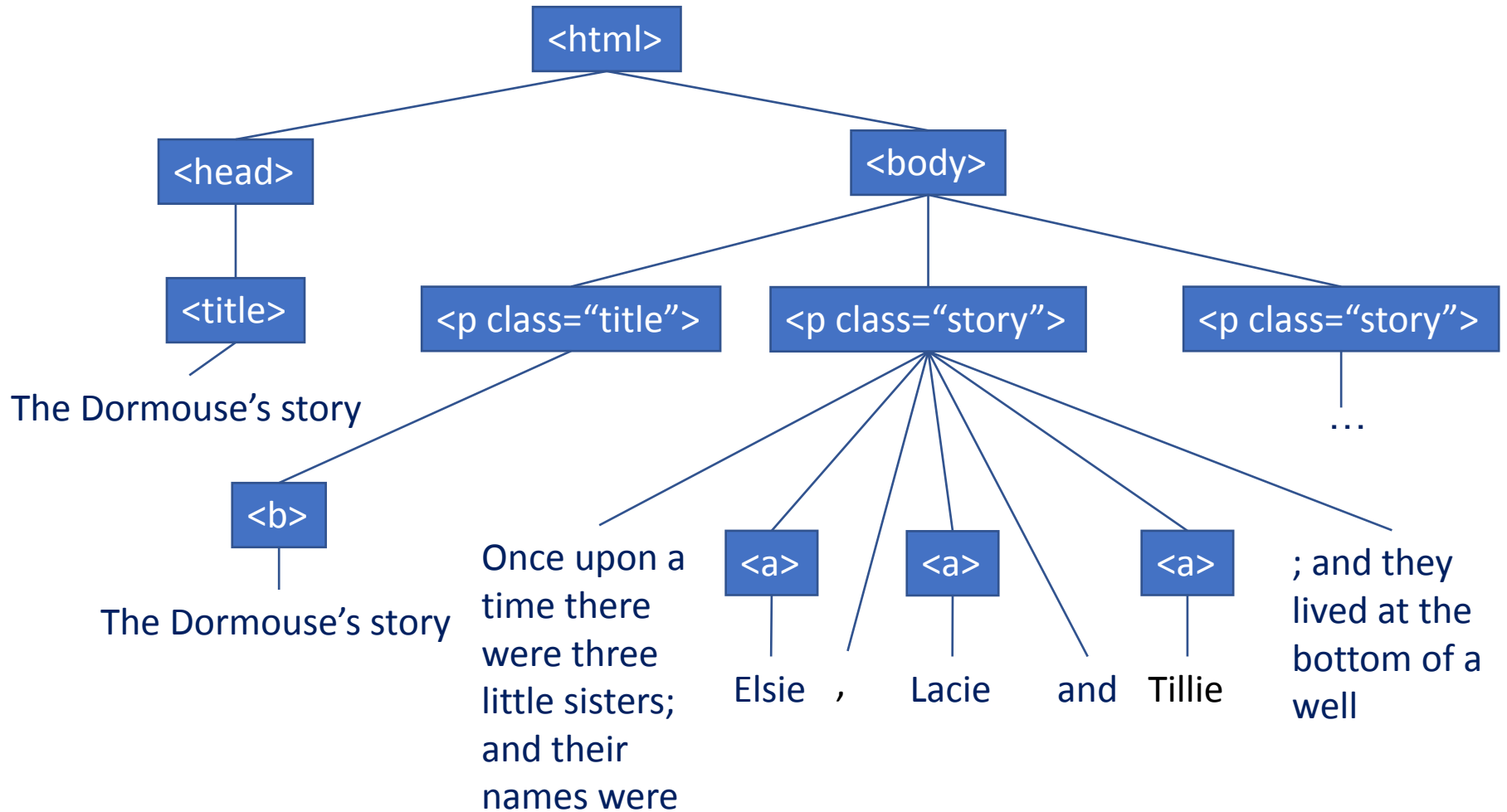
Beautiful Soup example – 2

Output from parser

```
from bs4 import BeautifulSoup
soup = BeautifulSoup(html_doc, 'html.parser')
```

```
<html>
<head>
  <title>
    The Dormouse's story
  </title>
</head>
<body>
  <p class="title">
    <b>
      The Dormouse's story
    </b>
  </p>
  <p class="story">
    Once upon a time there were three little sisters; and their names were
    <a class="sister" href="http://example.com/elsie" id="link1">
      Elsie
    </a>
    ,
    <a class="sister" href="http://example.com/lacie" id="link2">
      Lacie
    </a>
    and
    <a class="sister" href="http://example.com/tillie" id="link3">
      Tillie
    </a>
    ;
    and they lived at the bottom of a well.
  </p>
  <p class="story">
    ...
  </p>
</body>
</html>
>>>
```

Parse tree output



Beautiful Soup example – 3

Extracting just the text from the parse tree

```
print(soup.get_text())
```

```
>>> print(soup.get_text())
```

```
The Dormouse's story
```

```
The Dormouse's story
```

```
Once upon a time there were three little sisters; and their names were
```

```
Elsie,
```

```
Lacie and
```

```
Tillie;
```

```
and they lived at the bottom of a well.
```

```
...
```

```
>>> █
```

Beautiful Soup example – 3

Extracting just the text from the parse tree

```
print(soup.get_text())
```

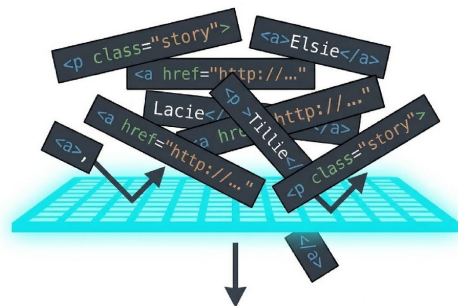
```
>>> print(soup.get_text())
```

```
The Dormouse's story
```

```
The Dormouse's story
```

```
Once upon a time there were three little sisters; and their names were  
Elsie,  
Lacie and  
Tillie;  
and they lived at the bottom of a well.  
...
```

```
>>>
```



```
print(soup.get_text())
```

`.get_text()` ignores all architectural tags and returns only the human-readable string data hidden inside the parse tree.

Beautiful Soup example – 4

Extract by tags: find_all

```
soup.find_all('a')  
# [<a class="sister" href="http://example.com/elsie" id="link1">Elsie</a>,  
# <a class="sister" href="http://example.com/lacie" id="link2">Lacie</a>,  
# <a class="sister" href="http://example.com/tillie" id="link3">Tillie</a>]
```

```
soup.find_all(["a", "b"])  
# [<b>The Dormouse's story</b>,  
# <a class="sister" href="http://example.com/elsie" id="link1">Elsie</a>,  
# <a class="sister" href="http://example.com/lacie" id="link2">Lacie</a>,  
# <a class="sister" href="http://example.com/tillie" id="link3">Tillie</a>]
```

```
for tag in soup.find_all(True):  
    print(tag.name)  
#html  
#head  
#title  
#body  
#p  
#b  
#p  
#a  
#a  
#a  
#p
```


Beautiful Soup example – 5

Extracting all the URLs, using `<a>` tags:

```
for link in soup.find_all('a'):  
    print(link.get('href'))  
# http://example.com/elsie  
# http://example.com/lacie  
# http://example.com/tillie
```

Finding elements, by attribute

```
soup.find(id="link3")  
# <a class="sister" href="http://example.com/tillie" id="link3">Tillie</a>
```

Beautiful Soup example – 6

Navigating the result, by tags

```
soup.title
# <title>The Dormouse's story</title>

soup.title.name
# u'title'

soup.title.string
# u'The Dormouse's story'

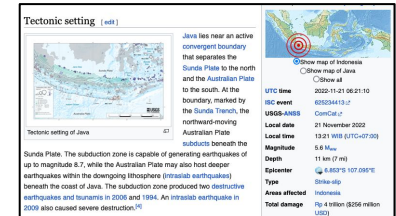
soup.title.parent.name
# u'head'

soup.p
# <p class="title"><b>The Dormouse's story</b></p>

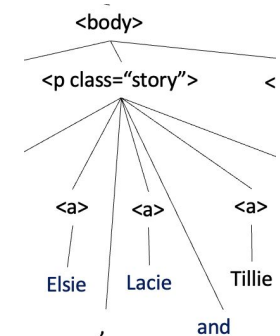
soup.p['class']
# u'title'
```

Scraping pipeline

1. Get your HTML or XML document
2. Decide which info you want to capture as data
3. Define your data fields in the database or spreadsheet
4. Use BeautifulSoup to parse the doc
5. Write BeautifulSoup extractor patterns to find the info you need and save it into their data fields
6. Voila!



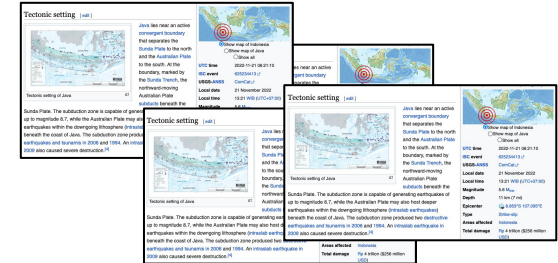
C	D	E	F
Name	ID	Age	Address



B	C	D	E	F	G
Angela	Brown	ambrown@unimelb.edu.au	jo.skidmore@unimelb.edu.au	CS	FABP
Tony	McKimmie	tony.mckimmie@unimelb.edu.au	stevie.troy@unimelb.edu.au	CS	ARTS
Diana	Dalton	diana.dalton@unimelb.edu.au	pmoylan@unimelb.edu.au	CS	FBE
Phil	Tudor	philip.tudor@unimelb.edu.au	jacinda.liau@unimelb.edu.au	CS	MGSE
Claire	Hooper	claire.hooper@unimelb.edu.au	rbrezac@unimelb.edu.au	CS	FEIT
Michelle	Chen	siwen.chen@unimelb.edu.au	giovanni.simele@unimelb.edu.au	CS	MLS
Andrew	Stott	andrew.stott@unimelb.edu.au	hayesc@unimelb.edu.au	CS	MDHS

Unstructured Text Pipeline

1. You crawled to collect pages
2. You scraped the desired parts out of the HTML
3. Now you are left with strings of text
4. You want to:
 - Break up sentences and words
 - Standardise words in various ways (no past tense, etc.)
 - Remove unnecessary material
5. And then, finally, you can process your text!



B	C	D	E	F	G
Angela	Brown	ambrown@unimelb.edu.au	"a lot of text..."	CS	FABP
Tony	McKimmie	tony.mckimmie@unimelb.edu.au		CS	ARTS
Diana	Dalton	diana.dalton@unimelb.edu.au		CS	FBE
Phil	Tudor	philip.tudor@unimelb.edu.au		CS	MGSE
Claire	Hooper	claire.hooper@unimelb.edu.au		CS	FET
Michelle	Chen	siwen.chen@unimelb.edu.au		CS	MLS
Andrew	Stott	andrew.stott@unimelb.edu.au		CS	MDHS
Michelle	Ward	m.ward@unimelb.edu.au		CS	SCI
Andrew	Hall	ahall@unimelb.edu.au		CS	FFAM



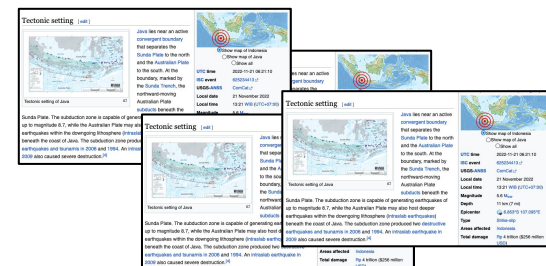
"On 21 November 2022, at 13:21 WIB, a M_{ww} 5.6 earthquake struck near Cianjur in West Java, Indonesia. At least 310 people died, 2,046 were injured, and 24 remain missing."



{ "On" "21/09/2022", "at", "01:21:00GMT", "a", " M_{ww} ", "5.6", "earthquake", "struck", ... }

Unstructured Image Pipeline

1. You crawled to collect pages
2. You scraped the desired parts out of the HTML
3. Now you are left with raw image files
4. You want to:
 - Resize or crop images
 - Standardise resolution and color channels
 - Remove noise or irrelevant regions
 - Extract useful features (edges, shapes, patterns)
5. And then, finally, you can process your image!



```
{ "id": "img_001",  
  "format": "RGB tensor",  
  "resolution": "256x256",  
  "label": "cat",  
  "features_vector": [...]  
}
```

Exercise: Crawling or Scraping?

1. Extracting all titles from a webpage?
2. Finding all pages of Wikipedia?

Exercise

https://en.wikipedia.org/wiki/Natural_language_processing

Using BeautifulSoup:

1. Extract all headers (“History”, “Approaches:”, etc.)?
2. List all NLP Tasks as provided under the “Common NLP tasks” header

The Complete Pipeline

